Table of Contents

Claude Code Orchestration — Requirements Specification

Version 3.0.0 · Updated 2026-05-23 · Author: Valverde · Supersedes "Claude Code Improvements.pdf" + v1, v2 of this spec

v3 changes summary (in response to the self-audit):

- Added meta-architecture-reviewer agent (the missing requirement A5 from the third message). Read-only; audits the plugin itself.
- Established reference/recommended-plugins.json as SINGLE SOURCE OF TRUTH for the 15-plugin list. CLAUDE.md, INSTALL, bootstrap, and plugin.json all reference it. No more drift across files.
- Consolidated the review pipeline to ONE policy reviewer + ONE bug-finder pass per unit (CLAUDE.md §5). No more 5-reviewer pile-up.
- Added graceful degradation (CLAUDE.md §20.5). Each agent now has documented fallback behavior when an official plugin isn't installed.
- Made Limitations.md + EdgeCases.md HARD GATES in implementation-planner. Cannot produce Plan-Final.md without 3+ limitations entries and human-confirmed edge cases per stage.
- Pre-implementation response template (CLAUDE.md §10) now includes Edge Cases and Limitations sections; orchestrator must wait for explicit confirmation of all three (Assumptions + Edge Cases + Parallelization).
- Added Postman MCP section (CLAUDE.md §20.6) — explicit install/use guidance.
- Documented the workbook-path decision (CLAUDE.md §20.7) — kept docs/ignored/workbooks/<slug>/ over original docs/temp/<team>/ with rationale.

v2 changes summary (preserved):

- §0 Authority: official plugins are tools, never replacements for custom agents
- §4 Agent inventory: +4 agents (architecture-enforcer, data-architect, comment-policy-checker, parallel-research-coordinator)
- §5 Skill inventory: +9 skills
- §6 New workflows: greenfield-vs-brownfield, plan-review-cycle, parallel-codebase-research-cycle, parallelization-decision, pr-merge-conflict-wait, assumption-validation-tests
- §9 MCP routing reframed: vendor-specific source FIRST, context7 generic fallback
- §10 New guardrails: SG10–SG14 (strengthened comment policy, parallel-decision, PR-merge-wait)
- §15 New: plan folder organization rule

- §16 New: mobile/remote coding section
- §17 New: local git hygiene story (.git/info/exclude)
- §18 New: 15 plugin dependencies declared

This document is the source of truth for what the orchestration architecture must do. Each requirement is numbered, has explicit acceptance criteria, and is traceable to the agent / skill / hook that implements it. Open questions are listed in §13.

---

1. Goals

- **G1** — Improve the coding cycle by decomposing it into an orchestrated set of specialist subagents instead of one monolithic agent.
- **G2** — Eliminate assumptions: every uncertainty becomes an explicit question to the human reviewer before code is written.
- **G3** — Keep all transient Claude-related artifacts (plans, workbooks, context snapshots) **out of the team repo**.
- **G4** — Guarantee that no commit, push, or PR carries Claude/Anthropic/AI attribution or process metadata (Jira IDs, branch names, plan references) in code comments or commit messages.
- **G5** — Enforce a mandatory **implementation feedback loop** (code → context refresh → review → test → docs) before any unit is considered done.
- **G6** — For bugs, enforce a **root-cause convergence loop** using independent investigations before claiming a root cause.
- **G7** — Distribute the architecture cleanly: one Claude Code plugin install + one small bootstrap script.

2. Non-goals

- **NG1** — This is **not** a team-wide engineering policy enforcement system. It is a personal/private overlay; the team's existing .claude/, CLAUDE.md, and .mcp.json are never touched.
- **NG2** — No agent will deploy, restart services, mutate production data, change feature flags, run database writes/migrations, force-push, rebase, reset, or delete branches without explicit human approval.
- **NG3** — No automated PR creation, commit, or push happens without an explicit human request.
- **NG4** — This architecture does **not** replace existing CI. It complements CI with pre-push validation.
- **NG5** — Agent teams (multi-agent direct communication) are **not** enabled by default; they are an opt-in escalation per §6.7.

3. Glossary

Term
Orchestrator

Subagent
Agent team
Skill
Hook
Workbook
Implementation Plan
Codebase Context
Module Context File

4. Agent inventory (functional requirements)

Each row is a hard requirement. The orchestrator's tools: Agent(…) allowlist enumerates exactly this set.

ID
A1
A2
A3
A4
A5
A6
A7
A8
A9
A10
A11
A12
A13
A14

A15

A16

A17

A18

A19

4.1 Hard constraints for all agents

- **HC1** Comments in code MUST NOT reference implementation plans, Jira tickets, branches, PRs, Claude, Anthropic, or AI. (Enforced by post-edit-policy-check.sh hook and by code-reviewer.)
- **HC2** No .md document is committed or pushed. (Enforced by Git pre-commit + pre-push hooks installed by the bootstrap.)
- **HC3** No commit/push includes Claude as co-author. (Enforced by Git commit-msg hook.)
- **HC4** No temporary scripts are committed. (Enforced by Git pre-commit hook patterns.)
- **HC5** No agent runs destructive git/DB/deploy commands without ALLOW_*=1 env vars. (Enforced by guard-git-and-dangerous-bash.sh hook.)
- **HC6** No agent edits team-visible config: repo/.claude/**, repo/CLAUDE.md, repo/.mcp.json — read-only unless explicitly approved. (Enforced by guard-edit-path-policy.sh hook.)
- **HC7** No agent mutates external systems via MCP without explicit approval. (Enforced by guard-mcp-mutations.sh hook with CLAUDE_ALLOW_MCP_MUTATION=1.)
- **HC8** Only the orchestrator may spawn the approved specialist set. Specialists may not spawn further specialists. (Enforced by guard-agent-spawn.sh hook.)

5. Skill inventory (procedural requirements)

ID

S1

S2

S3

S4

S5

S6

S7

S8

S9
S10
S11
S12
S13
S14
S15
S16
S17
S18
S19
S20
S21
S22
S23
S24
S25
S26
S27

6. Cross-cutting workflows

6.1 Implementation feedback loop (MANDATORY)

For every approved unit:

1. Coding agent implements **only** the approved unit.
2. codebase-contextualization refreshes docs/ignored/context/** for touched modules.
3. code-reviewer reviews every changed/added/deleted file against the approved stage, edge cases, architecture, comment policy, and existing patterns.
4. If review fails → coding fixes minimally → step 2 → step 3.
5. test-agent runs targeted tests first, then broader validation per the validation-matrix skill.
6. If validation fails → step 3 → step 5.
7. documentation-reviewer updates docs/ignored/** and removes stale/contradictory content.
8. Proceed only when review, validation, and docs gates pass — **or the human explicitly accepts the residual risk**.

Acceptance criterion (AC-6.1): the loop is encoded in skills/implementation-feedback-loop/SKILL.md and referenced by CLAUDE.md, engineering-orchestrator, coding-agent, code-reviewer, test-agent, documentation-reviewer, implementation-planner, implementation-planning, and validation-matrix.

6.2 No-assumptions discipline

Before planning or coding, the orchestrator must produce a "pre-implementation response" containing:

- Understanding · Confirmed constraints · Assumptions · Repo findings · Documentation/MCP findings · Relevant existing docs/ignored docs · Docs to create/update · Open questions · Status: waiting for explicit confirmation.

Implementation does not begin until the human explicitly confirms.

AC-6.2: CLAUDE.md §10 (Required pre-implementation response) and requirements-product-analyst enforce this.

6.3 Codebase Contextualization

Before any code is generated on a fresh branch **and** after every code change, a context snapshot is created/refreshed under docs/ignored/context/<module>/<Module>Context.md following the schema in §3 (Glossary → Module Context File).

The MCPs available for this step (used by parallel sub-investigations when budget allows):

- context7 — library docs
- serena — semantic code retrieval/refactoring/debugging
- codegraphcontext — code graph queries
- codanna — function discovery, call relationships
- tree-sitter — AST parsing
- codeql — static analysis
- srclight — code search

AC-6.3: codebase-contextualization skill exists and is referenced by the orchestrator and codebase-researcher. The skill never creates **/.module-context.md in the repo — only docs/ignored/context/**.

6.4 Workbook discipline

- Workbooks live under docs/ignored/workbooks/<team>/<task>/.
- Files are timestamped (filename or content header).
- README.md contains a metadata header + an activity log table (Date · Agent · Action).
- Workbooks are **gitignored** (covered by .git/info/exclude patterns installed by the bootstrap).

- Only durable, re-verified facts get promoted from workbook → long-lived docs/ignored/<Name>.md by documentation-refresh.
- Stale workbook content does **not** leak into long-lived docs.

AC-6.4: workbook-management and documentation-refresh skills exist. documentation-reviewer runs after every approved unit.

6.5 Pre-push validation matrix

The validation-matrix skill enumerates which checks are mandatory based on touched paths. Default rules (override per repo):

Touched path
app/web/src/**
src/ephai_intake/** or srv/**
schemas/queue-messages.json
Dockerfile / container build inputs

These rules are documented in the plugin's CLAUDE.md §6. They are intended as defaults — each consuming repo MUST customize.

AC-6.5: validation-matrix/SKILL.md documents these rules. pre-push-guardian enforces them on outgoing commits.

6.6 Branch + worktree workflow

- Branch name must be inferred from repo conventions (analyze recent branch names, CONTRIBUTING.md, PR title patterns).
- Base branch is the repo's default branch unless the human specifies otherwise.
- Branch creation **waits for human confirmation**.
- Worktree creation copies docs/ignored/ and .env* files per .worktreeinclude, then re-runs the bootstrap so the new tree gets Git hooks and core.hooksPath.
- Worktrees may need their own dependency install / virtualenv / artifact rebuild; the worktree-manager flags this in its handoff.

AC-6.6: branch-creation, worktree-handoff skills + worktree-manager agent + new-branch.sh / new-worktree.sh wrappers exist.

6.7 Subagents vs. agent teams decision

Per Claude Code docs:

Context	
Communication	
Coordination	
Best for	

Token cost

Default: subagents (cheaper, predictable). **Escalate to agent teams** only when: (a) two specialists need to compare hypotheses iteratively, (b) the task is collaborative reasoning (e.g. competing root-cause hypotheses), and (c) the human approves the token spend.

AC-6.7: agent-team-decision skill exists and is referenced by the orchestrator.

6.8 Root-cause convergence loop

For high-impact, production-facing, or uncertain bugs:

1. Capture original evidence from issue/logs/trace/CI/browser repro/user steps.
2. Split independent hypotheses by layer when useful.
3. Run independent investigations (parallel finder subagents, or an agent team) — **with explicit human approval**.
4. Compare findings; report convergence or divergence.
5. If findings diverge, do **not** force a conclusion; gather more evidence.
6. After fix, rerun the original reproduction/test/CI/browser/MCP check.

AC-6.8: root-cause-convergence skill exists, referenced by backend/frontend bug finders and the orchestrator.

6.9 Implementation plan reviews

Each implementation plan stage MUST include:

- **Assumptions** — explicit, each tagged "assumed" until human-confirmed.
- **Edge cases** — scenarios where the implementation can fall short, produce wrong results, or be wrongfully implemented.
- **Validation** — which tests / builds / manual checks gate this stage.
- **Limitations** — version-specific constraints, framework-specific behaviors, anything the stage will explicitly not cover.
- **Anti-overengineering reminder** — every stage instruction includes "do not over-engineer; if the simplest solution conflicts with the plan, surface it to the human."

Each plan runs multiple review rounds against the main requirements and against Context7/vendor-docs before implementation begins. Assumptions / edge cases must be human-confirmed.

AC-6.9: implementation-planning skill + implementation-planner agent enforce this.

6.10 Documentation hygiene

- Long-lived docs (docs/ignored/**, not workbooks) must be re-checked after every approved unit for stale/incorrect/missing info.
- Workbooks have timestamps and a progression order; long-lived docs do not — so the relationship is explicit: workbook → documentation-refresh → long-lived doc.

- Stale information in long-lived docs is **rewritten**, not appended to.
- Documentation reviewer is invoked automatically by the orchestrator after every approved unit **and** manually by the human via run-documentation-review skill.

AC-6.10: documentation-refresh skill + documentation-reviewer agent + CLAUDE.md §9 exist and align.

7. Output format contracts

7.1 Pre-implementation response (from §6.2)

Understanding
Confirmed Constraints
Assumptions
Repo Findings
Documentation/MCP Findings
Relevant Existing docs/ignored Docs
Docs To Create / Update
Open Questions
Status: waiting for explicit confirmation

7.2 Backend bug finder report

Finding Summary
Evidence Inspected
Reproduction
Root Cause (with confidence: high/medium/low)
Backend Bug Category (API / Auth/authz / Database / Queue/async / Integration / Config / Performance / Security / CI-test / Unknown)
Suggested Fix Plan
Regression Test Recommendation
Risks / Notes
Handoff Prompt For Bug Fixer

7.3 Frontend bug finder report

Finding Summary
Evidence Inspected
Reproduction
Root Cause (with confidence: high/medium/low)
Frontend Bug Category (JavaScript/runtime / Component rendering / SSR-hydration / Routing-navigation / State management / Forms / Network-API / Auth-session / CSS-layout / Accessibility / Performance / Build-test / Cross-browser-device / Visual-design regression / Unknown)
Suggested Fix Plan
Regression Test Recommendation
Risks / Notes
Handoff Prompt For Bug Fixer

7.4 PR creator output

Observed PR convention: <summary>
Title: <proposed title>
Body: <proposed body>

Base branch: <base>
Current branch: <branch>
Commands/checks used to inspect changes: <list>
Action: <only on explicit request, `gh pr create --base <base> --head <branch> --title "<title>" --body-file <temp-body-file>` (with --draft when requested)>

8. Distribution & install

Required deliverables:

- **D1** A Claude Code plugin (claude-code-orchestration) installable via /plugin install claude-code-orchestration@<marketplace> containing agents, skills, hooks, plugin-level .mcp.json, plugin-level settings.json (default agent), and CLAUDE.md.
- **D2** A marketplace manifest (marketplace/.claude-plugin/marketplace.json) so the plugin is discoverable via /plugin marketplace add.
- **D3** A repo bootstrap script (bootstrap/install_repo_bootstrap.sh) that is idempotent, prompts before changes, and installs only what plugins cannot: Git hooks, docs/ignored/, .git/info/exclude, .worktreeinclude, core.hooksPath, branch/worktree wrappers.
- **D4** An install guide (this document's companion INSTALL.md / .docx) covering prerequisites, both install paths (Git marketplace, local), daily use, verification, troubleshooting, and uninstall.

AC-8: All four deliverables exist in this package. The plugin frontmatter complies with Claude Code plugin restrictions (no per-agent hooks, mcpServers, permissionMode). Plugin-level hooks live in hooks/hooks.json. Plugin MCP config lives in .mcp.json at plugin root.

9. Required MCP integrations

MCP
context7
github
jira
atlassian (Jira+Confluence)
linear, asana, notion
playwright
chrome-devtools
storybook, chromatic
figma
sentry (observability)
serena, codanna, codegraphcontext, tree-sitter, codeql, srclight
database-readonly (per-project)

AC-9: .mcp.json documents which MCPs to install via plugin (recommended) and which require manual install. guard-mcp-mutations.sh blocks mutation-like calls unless CLAUDE_ALLOW_MCP_MUTATION=1.

10. Security & safety guardrails

ID
SG1
SG2
SG3
SG4
SG5
SG6
SG7
SG8
SG9

11. Acceptance test plan

Each test below is a smoke-level check; run after install to verify the system.

ID
T1
T2
T3
T4
T5
T6
T7
T8
T9
T10

12. Migration from the legacy private overlay

The previous install_private_overlay.sh approach is **deprecated**. To migrate:

1. Uninstall old user-level components:
   ```
   rm -rf ~/.claude/agents/<old-slug>
   rm -rf ~/.claude/skills/<old-slug>-*
   rm -rf ~/.claude-overlays/<old-slug>
   rm -f  ~/.local/bin/claude-<old-slug>
   ```

2. Install the new plugin per INSTALL.md Path A or B.
3. Re-run install_repo_bootstrap.sh for each affected repo. It is idempotent and will refresh the Git hook templates.

Legacy files to delete from disk: claude_code_architecture_pack*.zip, claude_code_private_overlay*.zip, the old Claude_Code_Private_Overlay_FINAL_Guide.{pdf,docx}, and the install_private_overlay.sh script.

13. Open questions & known limitations

ID
OQ1
OQ2
OQ3
OQ4
OQ5
OQ6
OQ7
OQ8

14. Traceability matrix (requirement → implementation)

Req
G1, A1–A19, HC1–HC8
S1–S27
SG1, SG2, SG4
SG5
SG6
HC1 (post-edit)
SG7, SG8 (Git layer)
G3, G4, 6.4, 6.6
6.7
6.8
D1
D2

D3

D4

Acceptance T1–T10

v2 supplement

§0 (new) Authority of the custom spec over official plugins

Official plugins (feature-dev, superpowers, code-review, context7, microsoft-docs, serena, github, atlassian, playwright, chrome-devtools-mcp, claude-md-management, commit-commands, claude-code-setup, pr-review-toolkit, frontend-design) are **tools used by the custom agents**. They never replace custom agents. If a plugin's workflow conflicts with a rule here, the rule here wins.

§4 (extension) New agents (v2)

ID
A20
A21
A22
A23

§5 (extension) New skills (v2)

ID
S28
S29
S30
S31
S32
S33
S34
S35
S36

§6 (extension) New workflows

6.11 Greenfield vs. brownfield

Default: brownfield (stricter). Detected by inspecting whether touched files exist.
Brownfield drift requires:

- "Drift: " declaration in implementation plan
- Human approval in writing

- Module Context File update

Greenfield: use feature-dev@claude-plugins-official's code-explorer + code-architect for design proposals; architecture-enforcer records the chosen design as the Module Context File.

AC-6.11: greenfield-vs-brownfield/SKILL.md exists. CLAUDE.md §8 references it.

6.12 Plan review cycle (multi-round convergence)

Plans accumulate stale content across revisions. The cycle:

- Round 1: 3 parallel reviewers (requirements / architecture-enforcer / feasibility) → individual review files.
- Round 2: consolidator merges → dedup → severity-rank.
- Round 3: planner revises to address blockers + majors → Plan-v(n+1).md.
- Round 4: re-review.
- Max 5 rounds. Only Plan-Final.md is implementable.

AC-6.12: plan-review-cycle/SKILL.md + architecture-enforcer/agent.md + new orchestrator workflow.

6.13 Parallel codebase research cycle

For research touching >3 modules. Decompose → ASK PARALLELIZATION → fan-out → consistency reviewer → gap-fill (max 3 rounds) → final synthesis. Each researcher writes its own timestamped file; final synthesis is the only artifact downstream agents read.

AC-6.13: parallel-research-coordinator/agent.md + parallel-codebase-research-cycle/SKILL.md.

6.14 Parallelization decision is opt-in

Default SERIALIZE. Before any fan-out: enumerate stages, declare dependency type (none/data-only/files-overlap/unknown), ASK the human. Record in Parallelization-<ts>.md.

AC-6.14: parallelization-decision/SKILL.md + CLAUDE.md §7 + orchestrator step 8.

6.15 Assumption-validation tests (pre/post)

Unless strictly sure, write a test that fails first. Implement. Re-run. The pre-test FAIL output and post-test PASS output are persisted as artifacts. Compare. Complements superpowers red-green-refactor TDD enforcement when installed.

AC-6.15: assumption-validation-tests/SKILL.md + CLAUDE.md §6.

6.16 PR merge-conflict wait

After pr-creator opens a PR, poll GH mergeable + mergeStateStatus + statusCheckRollup. CONFLICTING → STOP. BEHIND → recommend rebase. Failing required checks → report. PR is "submitted" only when MERGEABLE + CLEAN.

AC-6.16: pr-merge-conflict-wait/SKILL.md + pr-creator/agent.md updated to invoke it.

6.17 Official docs first (routing rule)

For any external behavior claim: pick the MOST-SPECIFIC source first. Microsoft → microsoft-docs. Atlassian → atlassian. GitHub → github. Browser → chrome-devtools-mcp + playwright. Generic → context7. No plugin available → vendor MCP / WebSearch. Cite the source. No claim from training memory alone.

AC-6.17: official-docs-first/SKILL.md + CLAUDE.md §3.

6.18 Comment / commit / PR metadata policy (4-layer enforcement)

Forbidden in code comments AND commit messages AND PR bodies:

- Ticket IDs (Jira-style [A-Z]{2,10}-\d+)
- Branch names (feature/…, fix/…, etc.)
- Implementation plan filenames (Plan*.md, Stage-*-*.md)
- Phase / stage references
- Workbook references / docs/ignored/ paths
- Claude / Anthropic / AI / GPT / Copilot attribution
- /

Enforced at:

1. post-edit-policy-check.sh hook (after-edit warning)
2. comment-policy-checker agent (pre-commit structured verdict)
3. pre-push-guardian agent (final pre-push gate)
4. Git pre-commit + commit-msg + pre-push hooks installed by bootstrap

Bypass with ALLOW_COMMENT_POLICY_BYPASS=1 per commit only (for legitimate string literals).

AC-6.18: comment-policy-checker/agent.md + updated post-edit-policy-check.sh + updated bootstrap hooks.

§9 (extension) MCP / plugin routing — v2

Plugins are preferred over raw MCPs when both exist. Routing by topic:

Topic
Microsoft / Azure / .NET
Atlassian / Jira / Confluence

GitHub API

Browser / DOM

Any other library/framework

Codebase

Postman

Google Cloud / AWS

AC-9.v2: .mcp.json has empty mcpServers (deferred to plugins). _recommendedPlugins in plugin.json lists all 15. CLAUDE.md §3 and §4 enforce the routing.

§10 (extension) New guardrails

ID
SG10
SG11
SG12
SG13
SG14
SG15
SG16

§15 (new) Plan folder organization

```
docs/ignored/implementation/<feature-slug>/
  README.md
  Requirements.md
  Plan-v1.md, Plan-v2.md, ..., Plan-Final.md
  Review-1-{requirements,architecture,feasibility}-<ts>.md
  Review-Consolidated-<n>-<ts>.md
  Parallelization-<ts>.md
  Limitations.md
  EdgeCases.md
  Stages/Stage-<n>-<title>.md
  Progress.md
  PostMortem.md
```

Slug: kebab-case, NO ticket IDs, NO branch names, NO team-internal identifiers.

AC-15: plan-folder-organization/SKILL.md defines the layout. implementation-planner enforces it.

§16 (new) Mobile / remote coding

Three viable options:

1. Claude.ai mobile app (Max/Team plans expose a Claude Code surface)
2. SSH into a workstation that runs Claude Code (Termius / Blink / Tailscale SSH)

3. Tailscale VPN + workstation

In all three, the agents, skills, hooks, and plugins live on the workstation — the phone is the I/O device.

§17 (new) Local git hygiene

Personal patterns go in .git/info/exclude, NEVER .gitignore. Bootstrap auto-populates standard patterns. gitignore-local-hygiene skill is invoked whenever Claude or the human creates a personal-only file. The user's pain story (untracked DPRD-3543 files following a checkout) is the canonical example.

AC-17: gitignore-local-hygiene/SKILL.md + bootstrap populates .git/info/exclude + CLAUDE.md §17.

§18 (new) Plugin dependencies

This plugin **recommends** (does not auto-install — that's the user's choice) these 15 official plugins. The bootstrap's --print-plugins flag prints the install commands. CLAUDE.md §21 names each.

context7 code-review feature-dev superpowers github atlassian playwright chrome-devtools-mcp serena claude-md-management commit-commands claude-code-setup pr-review-toolkit microsoft-docs frontend-design

Plus, manual MCP installs when needed:

- Postman MCP
- codegraphcontext, tree-sitter, codanna, codeql, srclight (no plugin yet)
- Vendor MCPs (Google Cloud, AWS, Stripe, Twilio, etc.) as needed

AC-18: _recommendedPlugins in plugin.json + INSTALL.md §3.1 + bootstrap --print-plugins flag.

Open questions / known limitations (v2 additions)

ID
OQ9
OQ10
OQ11
OQ12
OQ13
OQ14